Chapter 7 worked solutions

DS07-01 • Stability and inversion count

Passes are `(2,B),(5,A),(4,C),(2,D)`; `(2,B),(4,C),(5,A),(2,D)`; and `(2,B),(2,D),(4,C),(5,A)`. Strict inversions are A-B, A-C, A-D, C-D, totaling 4. Shifts are 1+1+2=4. Equal keys B and D do not form a strict inversion. With $\geq$ movement, D passes B, breaking stability. Shift count then no longer equals the strict inversion count.

DS07-02 • Three-way partition and stack depth

Initially lt=i=0, gt=6. Reading 4 advances i. Reading 2 swaps it to index 0 and gives lt=1, i=2. Reading 4 advances i=3. Reading 7 decrements gt to 5 and swaps indices 3 and 5, giving `[2,4,4,4,1,7]`, with i unchanged. Reading that 4 advances i. Reading 1 swaps it with lt=1, giving `[2,1,4,4,4,7]`, lt=2, i=gt=5. Partition is complete. Each actual recursive child is at most half the current nonequal region, so active sizes decay geometrically and stack depth is O(log n). Extreme splits still scan lengths n, n−1,… through the looped larger side, totaling $\Theta(n^2)$.

DS07-03 • The model behind the lower bound

Distinct keys have n! relative permutations requiring separate leaves. A binary comparison tree needs height at least $\log_2(n!)$. The inequality `n!≥(n/2)^(n/2)` gives Ω(n log n). Counting sort indexes directly by key and exceeds the comparison-only information model. Equal keys do not require distinguishing n! key orders, so this argument does not force n log n work on that single input. The lower bound remains a worst-case result for the general problem.

DS07-04 • A counterexample to unstable LSD passes

The stable units pass yields `31,11,22,14`. The tens pass must retain 11 before 14 and produce `11,14,22,31`. Reversing equal-tens order gives `14,11,22,31`, which is not sorted. With fixed key width w, $d \approx \text{ceil}(w/\log_2 b)$. Larger b reduces passes but increases counter initialization and storage per pass. Total time is Θ(d(n+b)). The useful base depends on n, key width, and memory constraints, not only the number of passes.

Chapter 7 worked solutions (continued)

Verification code appendix: sorting.cpp

```
#include <algorithm>
#include <cstddef>
#include <iostream>
#include <random>
#include <stdexcept>
#include <utility>
#include <vector>

void insertion_sort(std::vector<int>& values) {
        for (std::size_t i = 1; i < values.size(); ++i) {
                const int key = values[i];
                std::size_t position = i;
                while (position > 0 && values[position - 1] > key) {
                        values[position] = values[position - 1];
                        --position;
                }
                values[position] = key;
        }
}

void merge_range(std::vector<int>& values, std::vector<int>& buffer,
        std::size_t left, std::size_t right) {
        if (right - left < 2) {
                return;
        }
        const auto middle = left + (right - left) / 2;
        merge_range(values, buffer, left, middle);
        merge_range(values, buffer, middle, right);
        std::size_t a = left;
        std::size_t b = middle;
        for (std::size_t out = left; out < right; ++out) {
                if (a < middle && (b == right || values[a] <= values[b])) {
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 7 worked solutions (continued)

```
                        buffer[out] = values[a++];
                } else {
                        buffer[out] = values[b++];
                }
        }
        std::copy(buffer.begin() + static_cast<std::ptrdiff_t>(left),
                buffer.begin() + static_cast<std::ptrdiff_t>(right),
                values.begin() + static_cast<std::ptrdiff_t>(left));
}

void merge_sort(std::vector<int>& values) {
        std::vector<int> buffer(values.size());
        merge_range(values, buffer, 0, values.size());
}

void quick_range(std::vector<int>& values, std::size_t left, std::size_t right) {
        while (right - left > 1) {
                const int pivot = values[left + (right - left) / 2];
                std::size_t lower = left;
                std::size_t cursor = left;
                std::size_t greater = right;
                while (cursor < greater) {
                        if (values[cursor] < pivot) {
                                std::swap(values[lower++], values[cursor++]);
                        } else if (values[cursor] > pivot) {
                                std::swap(values[cursor], values[--greater]);
                        } else {
                                ++cursor;
                        }
                }
                if (lower - left < right - greater) {
                        quick_range(values, left, lower);
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 7 worked solutions (continued)

```
                        left = greater;
                } else {
                        quick_range(values, greater, right);
                        right = lower;
                }
        }
}

void quick_sort(std::vector<int>& values) {
        quick_range(values, 0, values.size());
}

struct Record {
        int key;
        int id;
};

void stable_insertion_sort(std::vector<Record>& records) {
        for (std::size_t i = 1; i < records.size(); ++i) {
                const Record item = records[i];
                std::size_t position = i;
                while (position > 0 && records[position - 1].key > item.key) {
                        records[position] = records[position - 1];
                        --position;
                }
                records[position] = item;
        }
}

void require(bool condition, const char* message) {
        if (!condition) {
                throw std::runtime_error(message);
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 7 worked solutions (continued)

```
        }
}

int main() {
        std::vector<std::vector<int>> cases{{}, {1}, {2, 2, 2}, {5, 2, 4, 2},
                {1, 2, 3, 4, 5}, {5, 4, 3, 2, 1}, {0, -5, 3, -5, 9}};
        std::mt19937 random(20261024);
        for (int trial = 0; trial < 180; ++trial) {
                std::vector<int> values(random() % 160);
                for (auto& value : values) {
                        value = static_cast<int>(random() % 41) - 20;
                }
                cases.push_back(values);
        }
        for (const auto& input : cases) {
                auto expected = input;
                std::sort(expected.begin(), expected.end());
                for (auto sorter : {insertion_sort, merge_sort, quick_sort}) {
                        auto actual = input;
                        sorter(actual);
                        require(actual == expected, "sort differs from reference
permutation");
                }
        }
        std::vector<Record> records{{5, 0}, {2, 1}, {4, 2}, {2, 3}};
        stable_insertion_sort(records);
        require(records[0].id == 1 && records[1].id == 3 && records[2].id == 2 &&
                records[3].id == 0, "equal-key stability was lost");
        std::cout << "PASS sorting: " << cases.size()
                << " cases for each of 3 algorithms; record stability\n";
}
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study